Intro to Building APIs

 (https://github.com/learn-co-curriculum/python-p4-intro-to-building-apis)  (https://github.com/learn-co-curriculum/python-p4-intro-to-building-apis/issues/new)

Learning Goals

- Build APIs to handle GET, POST, PATCH, and DELETE requests.

Key Vocab

- **Application Programming Interface (API)**: a software application that allows two or more software applications to communicate with one another. Can be standalone or incorporated into a larger product.
- **HTTP Request Method**: assets of HTTP requests that tell the server which actions the client is attempting to perform on the located resource.
- `GET` : the most common HTTP request method. Signifies that the client is attempting to view the located resource.
- `POST` : the second most common HTTP request method. Signifies that the client is attempting to submit a form to create a new resource.
- `PATCH` : an HTTP request method that signifies that the client is attempting to update a resource with new information.
- `DELETE` : an HTTP request method that signifies that the client is attempting to delete a resource.

Introduction

In the previous module, we learned about APIs. Prior to starting at Flatiron, you probably only used the internet for browsing, shopping, various productivity tasks, and so on. Now that you're building applications, APIs allow you to greatly expand their functionality without having to rebuild Facebook or Google Maps on your own. You've learned a bit about consuming APIs (accessing their resources); now it's time to build your own.

Flask is an ideal tool for building APIs. Because APIs are meant to provide a means of communication between machines, we don't need to get too fussy with appearances. Being a microframework, Flask doesn't include too many requirements for your frontend *or* your backend- you can quickly expose a database's resources on the internet with minimal configuration. Flask also makes it easy to specify which HTTP request methods each resource accepts: if you want a page to be readable but *not* modifiable, you can specify that the only acceptable method is `GET`. If you want to make a resource modifiable but not removable, you can specify that it does not accept `DELETE` requests.

All this being said, Flask being a microframework does not prevent you from making a beautiful frontend. In fact, it plays so nice with other frameworks that you don't even have to build your frontend with Python!

In this module you will:

- Configure an API that accepts `GET` requests.
- Add resources that accept `POST`, `PATCH`, and `DELETE` requests.
- Build a chatbot using a Flask API backend and a React frontend.

Resources

- **Flask - Pallets (https://flask.palletsprojects.com/en/2.2.x/)**
- **HTTP request methods - Mozilla (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods)**
- **What is an API? - MuleSoft (https://www.mulesoft.com/resources/api/what-is-an-api)**